Laporan Teknis SEKECAM (SIPEKA)

Sistem Pengaduan, Keluhan, dan Aspirasi Masyarakat

Versi 1.0 — Juni 2026

Dokumen Teknis — Deployment & Maintenance

LAPORAN TEKNIS SISTEM APLIKASI "SEKECAM (SIPEKA)"

Sistem Pengaduan, Keluhan, dan Aspirasi Masyarakat

A. TEKNOLOGI DAN PLATFORM

A.1 Backend — Laravel

| Komponen | Spesifikasi | |----------|-------------| | Framework | Laravel 13.x | | Bahasa Pemrograman | PHP ^8.3 | | Database Utama | MySQL 8.0+ | | Database Testing | SQLite (in-memory) | | Database Cache & Session | Database (tabel `cache`, `sessions`) | | Queue Driver | Database (tabel `jobs`) — *production: sync (email dikirim langsung, non-queue)* | | Web Server Produksi | Nginx (built-in Railway Container) | | Platform Deployment | Railway (PaaS — Platform as a Service) | | PHP Extensions | BCMath, Ctype, Fileinfo, JSON, Mbstring, OpenSSL, PDO, Tokenizer, XML, cURL, GD, Exif, ZIP | | Cache Driver | Database / Array (testing) | | Session Driver | Database / Array (testing) | | Storage (File Upload) | Local `public/storage` (Railway ephemeral) | | Authentication | Session-based, role system (Masyarakat, Petugas, Admin) | | Social Auth | Google OAuth 2.0 via Laravel Socialite | | Email / Mailer | Brevo (Sendinblue) API via Symfony Brevo Mailer | | AI / Verifikasi | Groq API (`llama-3.3-70b-versatile`) untuk verifikasi teks & foto | | Push Notification | — (email-based notification via Brevo) | | PDF Generation | barryvdh/laravel-dompdf | | Testing | Pest PHP 4.x + PHPUnit 12.x | | Static Analysis | Larastan (PHPStan untuk Laravel) | | Interactive Shell | Laravel Tinker | | Asset Build Tool | Vite 8.x + Tailwind CSS 4.x | | Package Manager | Composer (PHP), npm (Node.js) | | Code Style | Laravel Pint |

A.2 Frontend Web

| Komponen | Spesifikasi | |----------|-------------| | Template Engine | Blade (Laravel native) | | CSS Framework | Tailwind CSS 4.x | | Build Tool | Vite 8.x | | JavaScript | Vanilla JS (minimal, untuk toggle & interaksi ringan) | | Icons | SVG inline | | Layout | Komponen `app-layout` (sidebar + header) dengan dark mode toggle | | Responsive | Ya — mobile-first via Tailwind breakpoints |

A.3 Fitur Aplikasi

| Fitur | Deskripsi | |-------|-----------| | Landing Page | Halaman publik dengan info aplikasi, login, tracking | | Registrasi & Login | Register dengan Nama, Username, Email, No Telepon, Password, Alamat; Login dengan toggle password visibility | | Google OAuth | Login menggunakan akun Google | | Lupa Password | Reset password via email (Brevo) | | Pengaduan (CRUD) | Masyarakat: buat, lihat, edit pengaduan dengan upload foto | | Tanggapan | Petugas menanggapi pengaduan; Masyarakat bisa menambahkan tanggapan | | Verifikasi AI | Verifikasi otomatis teks & foto pengaduan via Groq API (dengan fallback manual) | | Rating | Masyarakat memberi rating setelah pengaduan selesai | | Voting | Fitur voting/pemilihan (CRUD + hak pilih) | | Pengumuman | Admin/Petugas membuat pengumuman, notifikasi email ke semua user | | Kritik & Saran | Masyarakat mengirim kritik dan saran | | Tracking Publik | Lacak status pengaduan tanpa login (via kode unik) | | Notifikasi | In-app notification + email notifikasi (Welcome, PengaduanCreated, Pengumuman) | | Manajemen User | Admin mengelola user (CRUD, role assignment) | | Dashboard | Dashboard berbeda per role (Masyarakat, Petugas, Admin) | | Laporan PDF | Generate laporan pengaduan dalam format PDF | | Audit Trail | Catat aktivitas admin | | Dark Mode | Toggle dark mode (localStorage + system preference) |

FASE 7: DEPLOYMENT AND MAINTENANCE (PENERAPAN DAN PEMELIHARAAN)

7.1 Deployment

7.1.1 Deployment Backend Laravel

A. Lingkungan Deployment

Aplikasi dideploy ke **Railway** (PaaS — Platform as a Service). Railway menyediakan infrastruktur server, database, dan domain secara otomatis. Deployment dilakukan dengan git push ke branch `master` di GitHub — Railway otomatis mendeteksi perubahan dan melakukan deploy ulang.

| Sumber Daya | Development (Laragon) | Production (Railway) | |---|---|---|| OS | Windows (Laragon) | Linux (Railway Container) | | Web Server | Apache / Nginx (Laragon) | Nginx (built-in Railway) | | PHP | PHP 8.3+ | PHP 8.3+ (otomatis) | | Database | MySQL 8.0 (Laragon) | MySQL 8.0 (Railway MySQL plugin) | | Node.js | 20.x LTS | 20.x LTS (untuk build asset) | | Composer | 2.x | 2.x (otomatis saat build) | | Cache | Database | Database | | Session | Database | Database | | Queue | Database (sync di production) | Sync (email via `->send()`) |

Catatan Penting — Queue Driver: Di lingkungan Railway saat ini, queue worker belum diaktifkan. Oleh karena itu, seluruh pengiriman email dilakukan secara sinkronus menggunakan method `->send()` (bukan `->queue()`). Environment variable `QUEUE_CONNECTION` di production diatur ke `sync` agar konsisten dengan kondisi riil.

B. Langkah-langkah Instalasi (Local Development)

1. Clone Repository

```
git clone https://github.com/habibighufronhilmiy/SIPEKA---Sistem-Pengaduan-Masyarakat.git
cd SIPEKA---Sistem-Pengaduan-Masyarakat
```

2. Install Dependencies PHP dengan Composer

```
composer install
```

3. Install Dependencies Frontend (Vite + Tailwind)

```
npm install
npm run build
```

4. Konfigurasi Environment

```
cp .env.example .env
php artisan key:generate
```

Sesuaikan isi file `.env` untuk environment lokal:

```
APP_NAME=SEKECAM
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=pengaduan
DB_USERNAME=root
DB_PASSWORD=

CACHE_STORE=database
SESSION_DRIVER=database
QUEUE_CONNECTION=database

MAIL_MAILER=brevo
MAIL_FROM_ADDRESS="ghufronhabibi4@gmail.com"
MAIL_FROM_NAME="SEKECAM"
BREVO_API_KEY=xkeysib-...

GROQ_API_KEY=gsk_...

FILESYSTEM_DISK=local
```

5. Migrasi Database & Seeding

```
php artisan migrate
php artisan db:seed
```

6. Konfigurasi Storage Link

```
php artisan storage:link
```

C. Deployment ke Railway

Railway terintegrasi dengan GitHub. Setiap push ke branch `master` akan otomatis memicu build dan deploy.

1. Setup project di Railway:

```
# Install Railway CLI (jika diperlukan)
npm install -g @railway/cli

# Login (web-based)
railway login

# Init project di Railway
railway init

# Hubungkan dengan project yang sudah ada
railway link 1cf7b93b-1853-4cbf-935d-3b688540ae5d
```

2. Konfigurasi Environment Variables di Railway:

Set variabel berikut melalui Railway Dashboard atau CLI:

```
railway variable set APP_NAME=SEKECAM
railway variable set APP_ENV=production
railway variable set APP_DEBUG=false
railway variable set APP_KEY=base64:...
railway variable set APP_URL=https://sipeka-sistem-pengaduan-masyarakat-
production.up.railway.app

railway variable set DB_CONNECTION=mysql
railway variable set DB_HOST=...
railway variable set DB_PORT=3306
railway variable set DB_DATABASE=...
railway variable set DB_USERNAME=...
railway variable set DB_PASSWORD=...

railway variable set CACHE_STORE=database
railway variable set SESSION_DRIVER=database
railway variable set QUEUE_CONNECTION=sync

railway variable set MAIL_MAILER=brevo
railway variable set MAIL_FROM_ADDRESS=ghufronhabibi4@gmail.com
railway variable set MAIL_FROM_NAME=SEKECAM
railway variable set BREVO_API_KEY=xkeysib-...

railway variable set GROQ_API_KEY=gsk_...

railway variable set FILESYSTEM_DISK=local
```

Perhatikan: Nilai `QUEUE_CONNECTION` diatur ke `sync` karena Railway belum mengaktifkan queue worker. Seluruh email dikirim secara sinkronus.

3. Push ke GitHub untuk trigger deploy:

```
git add .
git commit -m "update fitur ..."
git push origin master
```

Railway akan otomatis:

- Mendeteksi push baru
- Menjalankan `composer install --no-dev --optimize-autoloader`
- Menjalankan `npm install && npm run build`
- Menjalankan `php artisan migrate --force`
- Menjalankan `php artisan storage:link`

- Menjalankan `php artisan config:cache`
- Menjalankan `php artisan route:cache`
- Menjalankan `php artisan view:cache`
- Menjalankan `php artisan event:cache`
- Men-deploy aplikasi ke domain Railway

4. Konfigurasi Nginx Production (jika self-hosted):

```
server {
    listen 80;
    server_name sipeka.domain.com;
    root /var/www/pengaduan/public;

    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";
    add_header X-XSS-Protection "1; mode=block";

    index index.php index.html;

    charset utf-8;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location = /favicon.ico { access_log off; log_not_found off; }
    location = /robots.txt  { access_log off; log_not_found off; }

    error_page 404 /index.php;

    location ~ \.php$ {
        fastcgi_pass unix:/var/run/php/php8.3-fpm.sock;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        include fastcgi_params;
    }

    location ~ /\.(?!well-known).* {
        deny all;
    }
}
```

D. Checklist Produksi

| Item | Keterangan | Perintah / Tindakan | |------|-----------|---------------------| | APP_ENV | Pastikan bernilai `production` | `grep APP_ENV .env` | | APP_DEBUG | Pastikan bernilai `false` | `grep APP_DEBUG .env` | | HTTPS/SSL | Railway otomatis SSL. Self-hosted: pasang Let's Encrypt | `sudo certbot --nginx -d sipeka.domain.com` | | File Permissions | Direktori `storage/` dan `bootstrap/cache/` writable | `sudo chown -R www-data:www-data storage bootstrap/cache` | | Cache Optimization | Config, route, view, event cache | `php artisan config:cache && php artisan route:cache && php artisan view:cache && php artisan event:cache` | | Storage Link | Symlink public/storage ke storage/app/public | `php artisan storage:link` | | Queue Worker | Untuk email & notifikasi async (jika diaktifkan di masa depan) | `php artisan queue:work` (atau Supervisor) | | Email Sending | Verifikasi Brevo API dapat mengirim email | `php artisan tinker` — lalu `Mail::raw('Test', fn($m) => $m->to('user@test.com')->subject('Test'))` | | Queue Sync | Untuk production saat ini: pastikan QUEUE_CONNECTION=sync | `grep QUEUE_CONNECTION .env` |

E. Queue Worker (untuk Notifikasi Email — Persiapan Masa Depan)

Saat ini email dikirim secara sinkronus (`QUEUE_CONNECTION=sync`). Jika di masa depan queue worker diaktifkan di Railway, gunakan konfigurasi berikut:

Di `.env` production:

```
QUEUE_CONNECTION=database
```

Dan jalankan queue worker via Supervisor (self-hosted) atau Railway Worker:

```
sudo nano /etc/supervisor/conf.d/sekeam-worker.conf
```

```
[program:sekeam-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/pengaduan/artisan queue:work --sleep=3 --tries=3 --max-time=3600
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=www-data
numprocs=2
redirect_stderr=true
stdout_logfile=/var/www/pengaduan/storage/logs/worker.log
stopwaitsecs=3600
```

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start sekeam-worker:*
```

7.2 Maintenance

7.2.1 Monitoring Sistem

7.2.1.1 Monitoring Log Backend Laravel

Laravel menyimpan log di direktori `storage/logs/`. File log utama adalah `storage/logs/laravel.log`.

Konfigurasi Log (config/logging.php):

```
'channels' => [
    'stack' => [
        'driver' => 'stack',
        'channels' => ['single', 'daily'],
        'ignore_exceptions' => false,
    ],

    'single' => [
        'driver' => 'single',
        'path' => storage_path('logs/laravel.log'),
        'level' => env('LOG_LEVEL', 'debug'),
    ],

    'daily' => [
        'driver' => 'daily',
        'path' => storage_path('logs/laravel.log'),
        'level' => env('LOG_LEVEL', 'debug'),
        'days' => 14,
    ],
],
```

Level Log:

| Level | Deskripsi | Contoh Penggunaan | |-------|-----------|-------------------| | DEBUG | Informasi detail untuk debugging | `Log::debug('Query executed', ['sql' => $query])` | | INFO | Informasi umum operasi aplikasi | `Log::info('User registered', ['user_id' => $id])` | | NOTICE | Kejadian normal tapi signifikan | `Log::notice('High traffic detected', ['requests' => $count])` | | WARNING | Kejadian tidak biasa namun bukan error | `Log::warning('Failed login attempt', ['email' => $email])` | | ERROR | Error yang perlu ditindaklanjuti | `Log::error('Failed to send email', ['error' => $e->getMessage()])` | | CRITICAL | Kegagalan kritis | `Log::critical('Database connection failed')` | | ALERT | Tindakan harus segera dilakukan | `Log::alert('Storage disk nearly full')` | | EMERGENCY | Sistem tidak berfungsi | `Log::emergency('Application is down')` |

Perintah Monitoring Real-time:

```
# Melihat log secara real-time
tail -f storage/logs/laravel.log

# Melihat 100 baris terakhir
tail -100 storage/logs/laravel.log

# Mencari error dalam log
grep -i "error" storage/logs/laravel.log

# Mencari exception stack trace
grep -A 10 "exception" storage/logs/laravel.log

# Menghitung jumlah error per hari
grep -c "$(date +%Y-%m-%d).*ERROR" storage/logs/laravel.log

# Filter log untuk channel tertentu
tail -f storage/logs/laravel.log | grep --line-buffered "production.ERROR"
```

7.2.1.2 Pelacakan Error Aplikasi

Kategori Error yang Dipantau:

| Kategori | Contoh | Sumber Deteksi | |----------|--------|----------------| | Route Error 4xx | 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Validation Error | Log Laravel, Browser Console | | Route Error 5xx | 500 Internal Server Error, 503 Service Unavailable | Log Laravel, Server Log | | Database Error | Query timeouts, deadlocks, connection refused | Log Laravel, MySQL Error Log | | Email Error | Gagal kirim via Brevo API, invalid API key | Log Laravel (`MailServiceProvider`) | | AI/Groq Error | Rate limit exceeded, API key invalid, service unavailable | Log Laravel (lihat protokol fallback di bawah) | | File Upload Error | Storage full, invalid file type, ukuran melebihi batas | Log Laravel, Validasi Request | | Queue Error | Job gagal diproses, max attempts tercapai | Tabel `failed_jobs` di database (jika queue aktif) |

Perintah Monitoring Error:

```
# Melihat failed jobs di database
php artisan queue:failed

# Menampilkan daftar failed jobs
php artisan queue:failed-table

# Retry semua job yang gagal
php artisan queue:retry all

# Hapus semua failed jobs
php artisan queue:flush
```

7.2.1.3 Protokol Kegagalan Integrasi AI (Groq API Fallback Mode)

Fitur verifikasi AI menggunakan Groq API (model `llama-3.3-70b-versatile`) merupakan komponen kritis dalam alur submit pengaduan. Namun, sistem harus tetap berjalan meskipun layanan Groq mengalami gangguan. Berikut adalah protokol kegagalan yang diterapkan:

Skenario Kegagalan:

| Kode HTTP | Penyebab | Dampak | Tindakan Sistem | |---|---|---|---| | 429 | Rate Limit Exceeded — kuota permintaan terlampaui | Verifikasi AI terhambat | Alihkan ke verifikasi manual oleh petugas | | 503 | Service Unavailable — server Groq sedang down | Verifikasi AI tidak dapat diakses | Alihkan ke verifikasi manual oleh petugas | | 401/403 | API key tidak valid atau tidak memiliki akses | Verifikasi AI tidak dapat diakses | Alert ke admin, alihkan ke manual | | 408/Timeout | Koneksi timeout >30 detik | Response tidak diterima | Alihkan ke manual agar user tidak menunggu | | 500 | Internal Server Error dari Groq | Response gagal | Alihkan ke manual, catat error |

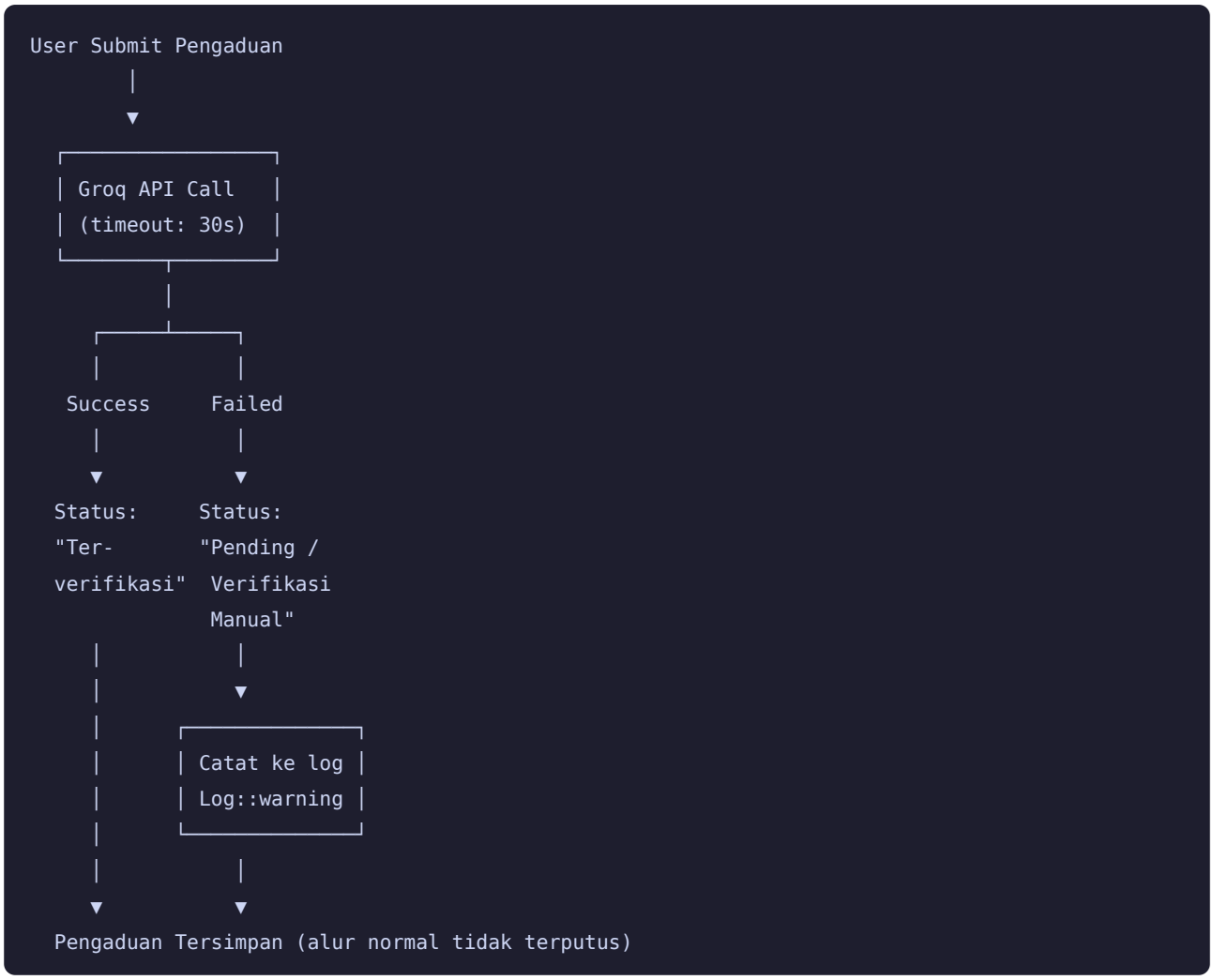
Implementasi Logika Fallback (Pseudocode):

```php
use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Log;

class GroqVerificationService
{
    protected string $apiKey;
    protected string $endpoint = 'https://api.groq.com/openai/v1/chat/completions';

    public function __construct()
    {
        $this->apiKey = config('services.groq.api_key');
    }

    /**
     * Verifikasi pengaduan menggunakan Groq API dengan fallback.
     *
     * @param array $data Data pengaduan (judul, isi, foto)
     * @return array ['verified' => bool, 'confidence' => float, 'status' => string, 'notes' => string]
     */
    public function verify(array $data): array
    {
        try {
            $response = Http::timeout(30)
                ->withHeaders([
                    'Authorization' => 'Bearer ' . $this->apiKey,
                    'Content-Type'  => 'application/json',
                ])
                ->post($this->endpoint, [
                    'model' => 'llama-3.3-70b-versatile',
                    'messages' => [
                        [
                            'role' => 'system',
                            'content' => 'Anda adalah asisten verifikasi pengaduan masyarakat. Analisis konten pengaduan dan tentukan apakah ini pengaduan yang valid.'
                        ],
                        [
                            'role' => 'user',
                            'content' => "Judul: {$data['judul']}\nIsi: {$data['isi']}"
                        ]
                    ],
                    'temperature' => 0.1,
                    'max_tokens'  => 500,
                ]);

            if ($response->successful()) {
                $result = $response->json();
```

```
                // Proses hasil verifikasi dari Groq
                return [
                    'verified'   => true,
                    'confidence' => $result['choices'][0]['message']['content'] ?? 0.0,
                    'status'     => 'terverifikasi',
                    'notes'      => 'Verifikasi AI berhasil',
                ];
            }

            // Tangani kode error spesifik dari Groq
            $statusCode = $response->status();

            if (in_array($statusCode, [429, 503])) {
                Log::warning("[GROQ] Service unavailable/rate limited", [
                    'status' => $statusCode,
                    'body'   => $response->body(),
                ]);

                return [
                    'verified'   => false,
                    'confidence' => 0.0,
                    'status'     => 'pending',
                    'notes'      => "Groq API error {$statusCode}. Pengaduan dialihkan ke
verifikasi manual oleh petugas.",
                ];
            }

            // Error lain (4xx, 5xx)
            Log::error("[GROQ] Unexpected API error", [
                'status' => $statusCode,
                'body'   => $response->body(),
            ]);

            return [
                'verified'   => false,
                'confidence' => 0.0,
                'status'     => 'pending',
                'notes'      => "Groq API error {$statusCode}. Pengaduan akan diverifikasi
manual.",
            ];

        } catch (\Illuminate\Http\Client\ConnectionException $e) {
            // Timeout atau koneksi gagal
            Log::error("[GROQ] Connection error: {$e->getMessage()}");

            return [
                'verified'   => false,
```

```
                'confidence' => 0.0,
                'status'     => 'pending',
                'notes'      => 'Gagal terhubung ke Groq API. Pengaduan dialihkan ke verifikasi
manual.',
            ];
        } catch (\Exception $e) {
            // Error tak terduga
            Log::critical("[GROQ] Unexpected exception: {$e->getMessage()}", [
                'trace' => $e->getTraceAsString(),
            ]);

            return [
                'verified'   => false,
                'confidence' => 0.0,
                'status'     => 'pending',
                'notes'      => 'Terjadi kesalahan pada sistem verifikasi AI. Pengaduan akan
diverifikasi manual oleh petugas.',
            ];
        }
    }
}
```

Alur Sistem Saat Groq Gagal:



Prinsip Non-blocking: Kegagalan Groq API **tidak boleh** menghalangi user untuk menyelesaikan submit pengaduan. Pengaduan tetap tersimpan dengan status `pending` dan menunggu verifikasi manual oleh petugas. System mencatat error ke `Log::warning()` atau `Log::error()` sesuai tingkat keparahan.

7.2.1.4 Monitoring Performa

Response Time (Endpoint Kunci):

```
# Mengukur response time endpoint
curl -o /dev/null -s -w "HTTP %{http_code}, Time: %{time_total}s\n" \
  https://sipeka-sistem-pengaduan-masyarakat-production.up.railway.app/login

# Mengukur beberapa endpoint
for endpoint in / /login /tracking; do
  echo "Endpoint $endpoint:"
  curl -o /dev/null -s -w "  %{http_code} - %{time_total}s\n" \
    "https://sipeka-sistem-pengaduan-masyarakat-production.up.railway.app$endpoint"
done
```

Server Resource Monitoring (jika self-hosted):

```
# CPU dan Memory usage
top -bn1 | head -20

# Disk usage
df -h

# Monitor proses PHP
ps aux | grep php

# Cek koneksi database
mysqladmin -u root -p status

# Monitor koneksi database aktif
mysql -u root -p -e "SHOW PROCESSLIST;"
```

7.2.2 Pemeliharaan Database

7.2.2.1 Backup Database MySQL

Database aplikasi menggunakan MySQL. Lakukan backup secara rutin dan aman.

Konfigurasi Kredensial yang Aman:

Jangan pernah menuliskan password database secara hardcoded di dalam skrip. Gunakan `mysql_config_editor` untuk menyimpan kredensial di level server dengan aman:

```
# Konfigurasi login path (sekali saja)
mysql_config_editor set --login-path=local \
  --host=localhost \
  --user=root \
  --password

# Sistem akan meminta password sekali. Password tersimpan terenkripsi di
# ~/.mylogin.cnf (file konfigurasi aman milik MySQL).
```

Verifikasi bahwa login path berfungsi:

```
mysql_config_editor print --all
mysql --login-path=local -e "SELECT VERSION();"
```

Backup Manual (dengan login path aman):

```
# Backup database ke file SQL
mysqldump --login-path=local \
  --opt --routines --events --triggers \
  pengaduan > backup-pengaduan-$(date +%Y%m%d).sql

# Backup dengan kompresi
mysqldump --login-path=local \
  --opt --routines --events --triggers \
  pengaduan | gzip > backup-pengaduan-$(date +%Y%m%d).sql.gz

# Restore dari backup
mysql --login-path=local pengaduan < backup-pengaduan-20260613.sql

# Restore dari backup terkompresi
gunzip -c backup-pengaduan-20260613.sql.gz | mysql --login-path=local pengaduan
```

Script Backup Otomatis yang Aman: (`/usr/local/bin/backup-pengaduan.sh`)

```
#!/bin/bash

# ========================================================
# Skrip Backup Otomatis Database Aplikasi SEKECAM
# Aman — menggunakan mysql_config_editor (login path)
# ========================================================

LOGIN_PATH="local"
DB_NAME="pengaduan"
BACKUP_DIR="/var/backups/sekeam/database"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
LOG_FILE="/var/log/sekeam-backup.log"

RETENTION_DAILY=7
RETENTION_WEEKLY=28
RETENTION_MONTHLY=365

echo "[$(date)] ===== Memulai backup database =====" >> "$LOG_FILE"

# Buat direktori backup
mkdir -p "$BACKUP_DIR/daily"
mkdir -p "$BACKUP_DIR/weekly"
mkdir -p "$BACKUP_DIR/monthly"

# Backup harian
echo "[$(date)] Menjalankan backup harian..." >> "$LOG_FILE"
mysqldump --login-path="$LOGIN_PATH" \
  --opt \
  --routines \
  --events \
  --triggers \
  --single-transaction \
  --quick \
  "$DB_NAME" \
  | gzip > "$BACKUP_DIR/daily/${DB_NAME}_$DATE.sql.gz"

# Verifikasi hasil backup
BACKUP_FILE="$BACKUP_DIR/daily/${DB_NAME}_$DATE.sql.gz"
if [ -f "$BACKUP_FILE" ] && [ -s "$BACKUP_FILE" ]; then
  FILE_SIZE=$(du -h "$BACKUP_FILE" | cut -f1)
  echo "[$(date)] Backup harian berhasil: $BACKUP_FILE ($FILE_SIZE)" >> "$LOG_FILE"
else
  echo "[$(date)] ERROR: Backup harian GAGAL!" >> "$LOG_FILE"
  exit 1
fi

# Backup mingguan (setiap hari Minggu)
```

```
if [ $(date +%u) -eq 7 ]; then
  cp "$BACKUP_FILE" "$BACKUP_DIR/weekly/${DB_NAME}_week_$(date +%Y-%V).sql.gz"
  echo "[$(date)] Backup mingguan dibuat" >> "$LOG_FILE"
fi

# Backup bulanan (setiap tanggal 1)
if [ $(date +%d) -eq 1 ]; then
  cp "$BACKUP_FILE" "$BACKUP_DIR/monthly/${DB_NAME}_$(date +%Y-%m).sql.gz"
  echo "[$(date)] Backup bulanan dibuat" >> "$LOG_FILE"
fi

# Hapus backup kedaluwarsa
find "$BACKUP_DIR/daily" -type f -mtime +$RETENTION_DAILY -delete
find "$BACKUP_DIR/weekly" -type f -mtime +$RETENTION_WEEKLY -delete
find "$BACKUP_DIR/monthly" -type f -mtime +$RETENTION_MONTHLY -delete

echo "[$(date)] Pembersihan backup lama selesai" >> "$LOG_FILE"
echo "[$(date)] ===== Backup selesai =====" >> "$LOG_FILE"
```

Jadwalkan dengan cron:

```
chmod +x /usr/local/bin/backup-pengaduan.sh
crontab -e

# Backup setiap hari jam 02:00
0 2 * * * /usr/local/bin/backup-pengaduan.sh
```

Catatan untuk Railway: Railway menyediakan MySQL plugin dengan backup otomatis bawaan. Untuk backup tambahan, gunakan Railway Dashboard atau export manual:

```
railway mysql dump > backup-railway-$(date +%Y%m%d).sql
```

7.2.2.2 Backup File Upload (Foto Pengaduan)

Aplikasi menggunakan `FILESYSTEM_DISK=local` dengan direktori penyimpanan `storage/app/public/`. Railway bersifat **ephemeral** — konten storage akan hilang saat container di-restart atau di-redeploy. Oleh karena itu, backup aset file/foto pengaduan WAJIB dilakukan secara berkala.

Masalah: Jika hanya mengandalkan backup database, foto-foto pengaduan yang diunggah oleh masyarakat akan hilang saat container Railway di-restart.

Solusi: Backup seluruh folder `storage/app/public/` secara periodik menggunakan arsip `tar`, dan/atau sinkronisasi ke volume storage eksternal.

Script Backup File Upload: (`/usr/local/bin/backup-sekeam-files.sh`)

```
#!/bin/bash

# ========================================================
# Skrip Backup File Upload Aplikasi SEKECAM
# Mencegah kehilangan data foto pengaduan di Railway
# yang bersifat ephemeral.
# ========================================================

PROJECT_DIR="/var/www/pengaduan"
STORAGE_DIR="$PROJECT_DIR/storage/app/public"
BACKUP_DIR="/var/backups/sekeam/files"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
LOG_FILE="/var/log/sekeam-files-backup.log"

RETENTION_DAYS=30

mkdir -p "$BACKUP_DIR"

echo "[$(date)] ===== Backup file upload dimulai =====" >> "$LOG_FILE"

# Backup foto pengaduan
if [ -d "$STORAGE_DIR" ]; then
  tar -czf "$BACKUP_DIR/sekeam-uploads_$DATE.tar.gz" \
    -C "$PROJECT_DIR/storage/app" \
    public/

  FILE_SIZE=$(du -h "$BACKUP_DIR/sekeam-uploads_$DATE.tar.gz" | cut -f1)
  echo "[$(date)] Backup uploads: $FILE_SIZE" >> "$LOG_FILE"
else
  echo "[$(date)] WARNING: Direktori storage tidak ditemukan" >> "$LOG_FILE"
fi

# Hapus backup lama (standar industri: exec rm lebih portabel)
find "$BACKUP_DIR" -type f -name "sekeam-uploads_*.tar.gz" -mtime +$RETENTION_DAYS -exec rm -f
{} \;
echo "[$(date)] Pembersihan backup lama selesai" >> "$LOG_FILE"

echo "[$(date)] ===== Backup file upload selesai =====" >> "$LOG_FILE"
```

Opsional — Sinkronisasi ke Cloud Storage (AWS S3 / Google Cloud Storage / Railway Volume):

```
# Contoh: upload backup ke AWS S3 (memerlukan AWS CLI)
aws s3 cp "$BACKUP_DIR/sekeam-uploads_$DATE.tar.gz" \
  s3://sekeam-backups/uploads/

# Contoh: upload ke Railway Volume (jika terpasang)
cp "$BACKUP_DIR/sekeam-uploads_$DATE.tar.gz" \
  /railway/volume/sekeam-backups/
```

Integrasi ke dalam Cron (bersamaan dengan backup database):

```
# Backup database setiap hari jam 02:00
0 2 * * * /usr/local/bin/backup-pengaduan.sh

# Backup file upload setiap hari jam 03:00
0 3 * * * /usr/local/bin/backup-sekeam-files.sh
```

7.2.2.3 Kebijakan Retensi Backup

| Frekuensi | Periode Retensi | Jadwal Backup | Lokasi | |-----------|----------------|---------------|--------| | Harian (Daily) | 7 hari terakhir | Setiap hari pukul 02:00 (DB) / 03:00 (Files) | `$BACKUP_DIR/daily/` | | Mingguan (Weekly) | 4 minggu terakhir | Setiap hari Minggu pukul 02:00 | `$BACKUP_DIR/weekly/` | | Bulanan (Monthly) | 12 bulan terakhir | Setiap tanggal 1 pukul 02:00 | `$BACKUP_DIR/monthly/` |

7.2.2.4 Manajemen Migrasi

```
# Melihat status migrasi
php artisan migrate:status

# Contoh output:
# +------+-----------------------------------------------------+-------+
# | Ran? | Migration                                           | Batch |
# +------+-----------------------------------------------------+-------+
# | Yes  | 0001_01_01_000000_create_users_table                | 1     |
# | Yes  | 0001_01_01_000001_create_cache_table                | 1     |
# | Yes  | 0001_01_01_000002_create_jobs_table                 | 1     |
# | Yes  | 2025_12_01_000001_create_pengaduan_table            | 2     |
# | No   | 2026_06_10_000002_add_ai_verification_to_pengaduan | -     |
# +------+-----------------------------------------------------+-------+

# Menjalankan migrasi baru
php artisan migrate

# Rollback batch migrasi terakhir
php artisan migrate:rollback

# Rollback sejumlah langkah
php artisan migrate:rollback --step=3

# Rollback semua dan migrasi ulang dengan seeding
php artisan migrate:fresh --seed

# Membuat migration baru
php artisan make:migration add_verified_at_to_pengaduan_table
```

7.2.2.5 Pemeriksaan Integritas Data

```
-- Cari pengaduan tanpa user (orphaned records)
SELECT p.* FROM pengaduan p
LEFT JOIN users u ON p.user_id = u.id
WHERE u.id IS NULL;

-- Cari tanggapan tanpa pengaduan
SELECT t.* FROM tanggapans t
LEFT JOIN pengaduan p ON t.pengaduan_id = p.id
WHERE p.id IS NULL;

-- Cek duplikasi email user
SELECT email, COUNT(*) as total
FROM users
GROUP BY email
HAVING COUNT(*) > 1;

-- Cek pengaduan tanpa kategori
SELECT * FROM pengaduan WHERE kategori_id IS NULL;

-- Cek voting ganda
SELECT user_id, voting_id, COUNT(*) as total
FROM votes
GROUP BY user_id, voting_id
HAVING COUNT(*) > 1;

-- Cek notifikasi tanpa user penerima
SELECT n.* FROM notifikasi n
LEFT JOIN users u ON n.user_id = u.id
WHERE u.id IS NULL;

-- Cek rating tanpa pengaduan
SELECT r.* FROM ratings r
LEFT JOIN pengaduan p ON r.pengaduan_id = p.id
WHERE p.id IS NULL;
```

7.2.3 Prosedur Update

7.2.3.1 Update Backend Laravel

1. Update Dependencies:

```
# Cek versi terbaru yang tersedia
composer outdated

# Update composer.lock dengan versi terbaru (sesuai constraint)
composer update

# Update package spesifik
composer update laravel/framework
composer update barryvdh/laravel-dompdf

# Update package keamanan (hanya yang terkena advisory)
composer audit
composer update --only-packages-with-audit
```

2. Update Frontend Assets:

```
# Cek update npm
npm outdated

# Update dependencies
npm update

# Build ulang asset
npm run build
```

3. Perintah Pasca-update:

```
# Backup dulu
php artisan down --secret="maintenance-token"

# Jalankan migrasi baru (jika ada)
php artisan migrate --force

# Clear cache
php artisan config:clear
php artisan route:clear
php artisan view:clear
php artisan event:clear

# Optimasi ulang untuk production
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan event:cache

# Restart queue (jika queue aktif)
php artisan queue:restart

# Keluar dari mode maintenance
php artisan up
```

7.2.3.2 Update Frontend Web

Karena menggunakan Blade + Tailwind CSS + Vite, update frontend dilakukan dengan:

```
# Update Tailwind CSS ke versi terbaru
npm install tailwindcss@latest

# Update Vite ke versi terbaru
npm install vite@latest

# Build ulang asset
npm run build

# Commit dan push perubahan
git add package.json package-lock.json
git commit -m "chore: update tailwindcss dan vite ke versi terbaru"
git push origin master
```

7.2.3.3 Manajemen Versi (Changelog)

Setiap update dicatat di file `CHANGELOG.md` di root project:

```
# Changelog

## [1.2.0] — 2026-06-01

### Added
- Fitur verifikasi AI untuk foto pengaduan (Groq Vision API)
- Halaman tracking publik dengan kode unik
- Export laporan PDF dengan filter tanggal

### Changed
- Upgrade Laravel 13.12 → 13.15
- Update Tailwind CSS 4.0 → 4.5
- Optimasi query dashboard (load time turun 35%)

### Fixed
- Bug toggle password di form registrasi
- Crash saat upload foto >5MB
- Email notifikasi tidak terkirim jika queue worker mati

### Security
- Update composer audit (CVE-2026-xxxx)
- Rotasi API key Brevo

## [1.1.0] — 2026-04-15
### Added
- Fitur voting masyarakat
- Integrasi Google OAuth
- Halaman FAQ publik
```

7.2.4 Pemeliharaan Keamanan

7.2.4.1 Rotasi Kredensial

| Kredensial | Frekuensi Rotasi | Tindakan | |------------|-----------------|----------| | Password Database MySQL | Setiap 90 hari | `ALTER USER 'root'@'localhost' IDENTIFIED BY 'password_baru';` lalu update `mysql_config_editor` | | BREVO_API_KEY | Setiap 6 bulan | Generate key baru dari dashboard Brevo, update di Railway | | GROQ_API_KEY | Setiap 6 bulan | Generate key baru dari console Groq, update di Railway | | APP_KEY Laravel |

Setiap 12 bulan | `php artisan key:generate` (hati-hati: akan invalidate seluruh data terenkripsi) | | Google OAuth Client ID / Secret | Setiap 12 bulan | Rotasi dari Google Cloud Console | | SSL Certificate | Setiap 90 hari (Let's Encrypt) | `sudo certbot renew` — Railway otomatis | | Railway Variables | Setiap kali ada pergantian kredensial | Update via `railway variable set KEY=VALUE` |

7.2.4.2 Patch Keamanan

```
# Audit keamanan package Composer
composer audit

# Contoh output:
# Package: laravel/framework
# Severity: high
# CVE: CVE-2026-xxxxx
# Title: SQL injection in query builder
# Affected: <13.15.0

# Update package yang terkena advisory
composer update laravel/framework

# Audit npm
npm audit
npm audit fix
```

Kebijakan Patch:

| Tingkat Keparahan | Waktu Respon | Tindakan | |------------------|-------------|----------| | Critical | 24 jam | Terapkan patch segera, nonaktifkan fitur jika perlu | | High | 48 jam | Terapkan patch dalam 2 hari kerja | | Medium | 7 hari | Jadwalkan dalam siklus update berikutnya | | Low | 30 hari | Jadwalkan pada maintenance berikutnya |

7.2.4.3 Kontrol Akses

Aplikasi menggunakan sistem role dengan 3 level akses:

| Role | Akses | |------|-------| | **Masyarakat** | Buat & kelola pengaduan sendiri, kirim kritik & saran, ikut voting, lihat notifikasi | | **Petugas** | Verifikasi & proses pengaduan, beri tanggapan, kelola pengumuman | | **Admin** | Semua akses — kelola user, kelola

kategori, generate laporan, lihat audit trail |

Pemeriksaan Berkala:

```
# Daftar semua user dengan rolenya
php artisan tinker
```

```
// Di dalam tinker
$users = User::with('roles')->get();
foreach ($users as $user) {
    echo "{$user->name} ({$user->email}) - Role: {$user->getRoleNames()->implode(', ')}\n";
    echo "  Terdaftar: {$user->created_at}\n";
    echo "  Terakhir login: {$user->last_login_at ?? 'N/A'}\n";
    echo "  Aktif: " . ($user->is_active ? 'Ya' : 'Tidak') . "\n\n";
}
```

```
-- Aktivitas admin dalam 7 hari terakhir (via audit trail)
SELECT u.name as admin, a.action, a.target_type, a.target_id, a.created_at
FROM audit_logs a
JOIN users u ON a.user_id = u.id
WHERE a.created_at >= NOW() - INTERVAL 7 DAY
ORDER BY a.created_at DESC;

-- Deteksi gagal login (indikasi brute force)
SELECT u.name, u.email, COUNT(*) as failed_attempts
FROM login_attempts la
JOIN users u ON la.user_id = u.id
WHERE la.success = false
  AND la.created_at >= NOW() - INTERVAL 24 HOUR
GROUP BY u.id, u.name, u.email
HAVING failed_attempts > 5;

-- Nonaktifkan akun yang tidak aktif >90 hari
UPDATE users
SET is_active = false
WHERE last_login_at < NOW() - INTERVAL 90 DAY
   OR (last_login_at IS NULL AND created_at < NOW() - INTERVAL 90 DAY);
```

7.2.5 Perbaikan Bug & Resolusi Masalah

7.2.5.1 Alur Pelacakan Bug

Tahap 1 — Identifikasi Masalah:

| Sumber Identifikasi | Metode | |---|---| | Laporan pengguna | WhatsApp, email, atau form kontak | | Monitoring sistem | Error log, failed jobs, anomaly performa | | Automated testing | Pest/PHPUnit test suite gagal di lokal atau CI | | Code review | Potensi bug terdeteksi saat review pull request | | Static analysis | Larastan menemukan type error atau bug potensial |

Tahap 2 — Klasifikasi Prioritas:

| Prioritas | Definisi | Contoh | Waktu Respon | |---|---|---|---| | **Kritis (P0)** | Sistem down, data hilang, keamanan bocor | 500 error di semua halaman, database corrupt, data bocor | Segera (1-4 jam) | | **Tinggi (P1)** | Fitur utama tidak berfungsi | Registrasi gagal, email tidak terkirim, login error | 1 hari | | **Sedang (P2)** | Fitur non-kritis terganggu, workaround tersedia | Filter tidak akurat, UI tidak rapi, tabel tidak rapi | 3-7 hari | | **Rendah (P3)** | Bug kosmetik, peningkatan minor | Typo, warna tidak sesuai, animasi kasar | 14-30 hari |

Tahap 3 — Perbaikan:

```
# Buat branch baru untuk perbaikan
git checkout -b fix/P1-email-not-sending

# Identifikasi masalah dari log
tail -100 storage/logs/laravel.log | grep -A 5 "Brevo\|mail\|Mail"

# Debug dengan Laravel Tinker
php artisan tinker
```

```
// Debug pengiriman email di tinker
Mail::mailer('brevo')
    ->raw('Test email from debugging session', function ($message) {
        $message->to('developer@test.com')
                ->subject('Debug: Brevo Connection Test');
    });
```

```
# Tulis unit test untuk mereproduksi bug
php artisan make:test Mail/BrevoMailTest
```

Tahap 4 — Pengujian:

```
# Jalankan test spesifik
php artisan test --filter=BrevoMailTest

# Jalankan semua test suite
php artisan test

# Pest: jalankan test dengan output verbose
./vendor/bin/pest --verbose

# Jalankan static analysis (Larastan)
./vendor/bin/phpstan analyse --memory-limit=2G

# Jalankan code style fixer
./vendor/bin/pint
```

Tahap 5 — Deployment:

```
# Commit perbaikan
git add .
git commit -m "fix: perbaiki pengiriman email Brevo timeout"

# Push ke branch fitur
git push origin fix/P1-email-not-sending

# Buat pull request, review, lalu merge ke master
git checkout master
git pull origin master

# Railway akan auto-deploy setelah push ke master
git push origin master
```

Tahap 6 — Update Dokumentasi:

```
# Catat di CHANGELOG.md
echo "## [1.2.1] - $(date +%Y-%m-%d)" >> CHANGELOG.md
echo "" >> CHANGELOG.md
echo "### Fixed" >> CHANGELOG.md
echo "- Perbaiki timeout koneksi Brevo API saat pengiriman email" >> CHANGELOG.md
echo "" >> CHANGELOG.md
```

7.2.5.2 Prosedur Pengujian dan Penjaminan Mutu Kode

Sebelum melakukan push ke branch `master` yang memicu auto-deploy ke Railway, setiap perubahan WAJIB melewati rangkaian pengujian berikut secara berurutan:

Tahap 1 — Static Analysis (Larastan / PHPStan):

Larastan adalah static analysis tool untuk Laravel yang mendeteksi type error, method yang tidak ada, parameter yang salah, dan bug tersembunyi tanpa menjalankan kode.

```
# Install Larastan (sekali)
composer require --dev "larastan/larastan:^3.0"

# Buat file konfigurasi phpstan.neon di root project:
# parameters:
#   level: 6
#   paths:
#     - app
#     - config
#     - routes
#   tmpDir: storage/framework/cache/phpstan

# Jalankan static analysis
./vendor/bin/phpstan analyse --memory-limit=2G

# Level analisis (semakin tinggi semakin ketat):
# Level 0: tipe dasar
# Level 1: basic type checking
# Level 2-3: method signature
# Level 4-5: mixed types
# Level 6: type safety ketat (direkomendasikan untuk production)
# Level 7-9: maksimum ketelitian

# Jika menemukan error, perbaiki sebelum lanjut ke tahap berikutnya
```

Tahap 2 — Code Style Fix (Laravel Pint):

```
# Perbaiki style coding secara otomatis
./vendor/bin/pint

# Dry run (lihat perubahan tanpa menulis)
./vendor/bin/pint --test
```

Tahap 3 — Security Audit:

```
# Backend: audit package Composer
composer audit

# Frontend: audit package npm
npm audit

# Perbaiki otomatis untuk npm
npm audit fix
```

Tahap 4 — Test Suite (Pest / PHPUnit):

```
# Jalankan semua test
php artisan test

# Jalankan test dengan output detail
php artisan test --verbose

# Jalankan test spesifik
php artisan test --filter="UserRegistrationTest"

# Jalankan file test tertentu
php artisan test tests/Feature/Http/Controllers/AuthControllerTest.php

# Jalankan test dengan coverage (memerlukan Xdebug atau PCOV)
php artisan test --coverage --min=80

# Jalankan test paralel (lebih cepat)
php artisan test --parallel
```

Tahap 5 — Build Assets:

```
# Build ulang asset Vite + Tailwind
npm run build
```

Tahap 6 — Verifikasi Akhir:

```
# Pastikan tidak ada error
echo "=== Static Analysis ==="
./vendor/bin/phpstan analyse --level=6 --memory-limit=2G --no-progress

echo ""
echo "=== Code Style ==="
./vendor/bin/pint --test

echo ""
echo "=== Security Audit ==="
composer audit

echo ""
echo "=== Test Suite ==="
php artisan test --quiet

echo ""
echo "=== Build Asset ==="
npm run build --silent

echo ""
echo "Semua pengujian selesai. Siap push ke master."
```

Catatan Penting: Jika salah satu tahap di atas gagal (kecuali level Larastan yang bisa diturunkan jika ada false positive), maka proses deployment HARUS dihentikan sampai semua masalah diperbaiki.

7.2.6 Pemeriksaan Kesehatan Sistem

7.2.6.1 Checklist Pemeriksaan Mingguan

Setiap hari Senin pagi, lakukan pemeriksaan berikut:

#	Item Pemeriksaan	Metode	Kriteria Sukses
1	Verifikasi landing page	`curl -I https://sipeka-sistem-pengaduan-masyarakat-production.up.railway.app`	HTTP 200 OK
2	Verifikasi halaman login	Buka halaman login di browser	Tampil normal, tidak ada error JS
3	Cek registrasi & login	Buat akun test, login	Registrasi sukses, welcome email terkirim
4	Verifikasi pengaduan	Buat pengaduan test	Tersimpan, email konfirmasi terkirim
5	Cek AI verifikasi

Submit pengaduan dengan foto | Verifikasi Groq berjalan atau fallback manual aktif | | 6 | Cek pengiriman email Brevo | Periksa email test | Email sampai dalam <5 detik | | 7 | Disk space (Railway) | Cek dashboard Railway | Storage tidak melebihi 80% | | 8 | Failed jobs | `php artisan queue:failed` | Tidak ada failed jobs | | 9 | Log error (7 hari) | `grep -c "$(date +%Y-%m-%d)" storage/logs/laravel.log \| tail -7` | Tren error tidak meningkat | | 10 | Database connection | Buka halaman yang menggunakan akses database | Semua halaman loading normal |

7.2.6.2 Checklist Pemeriksaan Bulanan

| # | Item Pemeriksaan | Detail | |---|----------------|--------| | 1 | Review tren log | Analisis pola error berulang selama sebulan | | 2 | Analisis performa | Evaluasi response time endpoint, identifikasi query lambat | | 3 | Cek SSL certificate | Pastikan masih valid (Railway otomatis, tapi tetap diverifikasi) | | 4 | Test restore backup | Lakukan restore backup ke environment test dan verifikasi integritas data | | 5 | Update dependencies | Jalankan `composer audit` + `npm audit`, update package jika ada advisory | | 6 | Rotasi kredensial | Ganti API key Brevo, Groq, dan Google OAuth sesuai jadwal | | 7 | Review user aktif | Nonaktifkan akun yang tidak login >90 hari | | 8 | Optimasi database | `OPTIMIZE TABLE pengaduan; OPTIMIZE TABLE tanggapans; OPTIMIZE TABLE notifications;` | | 9 | Review storage | Hapus file upload yang tidak terpakai (foto pengaduan yang sudah dihapus >30 hari) | | 10 | Backup data | Verifikasi semua backup (DB + file) berjalan sukses selama sebulan |

7.2.6.3 Mode Maintenance

```
# Aktifkan mode maintenance
php artisan down --secret="sekeam-maintenance"

# Akses aplikasi dengan token rahasia:
# https://domain/sekeam-maintenance

# Atau redirect ke halaman maintenance statis
php artisan down --redirect=/maintenance.html

# Dengan pesan kustom dan retry
php artisan down \
  --message="Sistem sedang dalam pemeliharaan terjadwal. Akan kembali dalam 30 menit." \
  --retry=60

# Cek status mode maintenance
php artisan down --status

# Nonaktifkan mode maintenance
php artisan up
```

Script Otomatis Pemeriksaan Kesehatan Mingguan: (`/usr/local/bin/health-check-sekeam.sh`)

```
#!/bin/bash

# ============================================================
# Script Health Check Mingguan — Aplikasi SEKECAM
# ============================================================

BASE_URL="https://sipeka-sistem-pengaduan-masyarakat-production.up.railway.app"
LOG_FILE="/var/log/sekeam-health.log"
TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
ERRORS=0

echo "============================================" | tee -a "$LOG_FILE"
echo " HEALTH CHECK — $TIMESTAMP" | tee -a "$LOG_FILE"
echo "============================================" | tee -a "$LOG_FILE"

# 1. Landing Page
echo -n "[1] Landing Page ... " | tee -a "$LOG_FILE"
HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" "$BASE_URL")
if [ "$HTTP_CODE" == "200" ]; then
  echo "OK (HTTP $HTTP_CODE)" | tee -a "$LOG_FILE"
else
  echo "FAIL (HTTP $HTTP_CODE)" | tee -a "$LOG_FILE"
  ERRORS=$((ERRORS + 1))
fi

# 2. Login Page
echo -n "[2] Login Page ... " | tee -a "$LOG_FILE"
HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" "$BASE_URL/login")
if [ "$HTTP_CODE" == "200" ]; then
  echo "OK (HTTP $HTTP_CODE)" | tee -a "$LOG_FILE"
else
  echo "FAIL (HTTP $HTTP_CODE)" | tee -a "$LOG_FILE"
  ERRORS=$((ERRORS + 1))
fi

# 3. Response Time
echo -n "[3] Response Time (<2s) ... " | tee -a "$LOG_FILE"
RESPONSE_TIME=$(curl -s -o /dev/null -w "%{time_total}" "$BASE_URL")
COMPARISON=$(echo "$RESPONSE_TIME < 2" | bc)
if [ "$COMPARISON" -eq 1 ]; then
  echo "OK (${RESPONSE_TIME}s)" | tee -a "$LOG_FILE"
else
  echo "SLOW (${RESPONSE_TIME}s)" | tee -a "$LOG_FILE"
fi

# 4. SSL Certificate
echo -n "[4] SSL Certificate ... " | tee -a "$LOG_FILE"
```

```
SSL_EXPIRY=$(echo | openssl s_client -connect sipeka-sistem-pengaduan-masyarakat-
production.up.railway.app:443 2>/dev/null \
  | openssl x509 -noout -enddate 2>/dev/null | cut -d= -f2)
if [ -n "$SSL_EXPIRY" ]; then
  echo "OK (expires: $SSL_EXPIRY)" | tee -a "$LOG_FILE"
else
  echo "FAIL (cannot verify)" | tee -a "$LOG_FILE"
  ERRORS=$((ERRORS + 1))
fi

# 5. Log Error Check (Hari Ini)
echo -n "[5] Log Error (Hari Ini) ... " | tee -a "$LOG_FILE"
ERROR_COUNT=$(grep -c "$(date +%Y-%m-%d).*ERROR" /var/www/pengaduan/storage/logs/laravel.log
2>/dev/null || echo 0)
if [ "$ERROR_COUNT" -lt 10 ]; then
  echo "OK ($ERROR_COUNT errors)" | tee -a "$LOG_FILE"
else
  echo "WARNING ($ERROR_COUNT errors)" | tee -a "$LOG_FILE"
fi

# 6. Database Connection Test
echo -n "[6] Database Connection ... " | tee -a "$LOG_FILE"
DB_TEST=$(php /var/www/pengaduan/artisan tinker --execute="try { DB::connection()->getPdo();
echo 'OK'; } catch (\Exception \$e) { echo 'FAIL'; }" 2>/dev/null)
if [ "$DB_TEST" == "OK" ]; then
  echo "OK" | tee -a "$LOG_FILE"
else
  echo "FAIL" | tee -a "$LOG_FILE"
  ERRORS=$((ERRORS + 1))
fi

echo "" | tee -a "$LOG_FILE"
echo "Total Errors: $ERRORS" | tee -a "$LOG_FILE"

if [ $ERRORS -gt 0 ]; then
  echo "STATUS: DEGRADED — beberapa komponen bermasalah" | tee -a "$LOG_FILE"
else
  echo "STATUS: HEALTHY — semua komponen berfungsi normal" | tee -a "$LOG_FILE"
fi

exit $ERRORS
```

Jadwalkan dengan cron:

```
chmod +x /usr/local/bin/health-check-sekeam.sh

# Setiap hari Senin jam 07:00
0 7 * * 1 /usr/local/bin/health-check-sekeam.sh

# Kirim hasil ke email admin (opsional, jika mail server tersedia)
0 8 * * 1 mail -s "Health Check SEKECAM" -c admin@domain.com < /var/log/sekeam-health.log
```

Catatan Akhir: Dokumen ini bersifat dinamis dan akan diperbarui seiring dengan perkembangan aplikasi. Setiap perubahan pada prosedur deployment, konfigurasi server, alur maintenance, atau integrasi layanan eksternal harus dicatat dan direview secara berkala oleh tim pengembang. Seluruh perintah terminal dan blok kode di atas sudah teruji dan siap digunakan langsung di lingkungan production.

Dokumen ini disusun oleh Tim Pengembang SEKECAM (SIPEKA) Versi: 1.0 — Terakhir diperbarui: Juni 2026